AI Interview Practice ? Project Brief

A concise overview of the product, implementation, and deployment model.

Project Overview

- AI Interview Practice is a web application designed to help candidates rehearse interviews with AI-driven questions, live feedback, and structured practice sessions.
- It combines a modern frontend, a Node/Express backend, real-time communication, and database persistence for a smooth practice workflow.

Purpose

- Provide a realistic interview environment without requiring a human interviewer.
- Help users build confidence through repeated practice, instant responses, and session history.
- Support scalable, cloud-hosted access for individual learners and interview preparation workflows.

Core Features

- AI-generated interview prompts tailored to the selected role or topic.
- Real-time interaction using sockets for responsive question/answer flow.
- User session handling, history tracking, and persisted practice records.
- Feedback-oriented experience for iterative improvement.
- Clean UI for starting sessions, answering questions, and reviewing outcomes.

Tech Stack

- Frontend: React-based client app (Vite-style structure expected in modern Node projects).
- Backend: Node.js with Express for REST APIs and server-side logic.
- Real-time: Socket.IO for live interview messaging/events.
- Database: MongoDB Atlas for cloud data persistence.
- Deployment: Vercel for frontend and Render for backend services.

Architecture

- Client-first workflow: the browser app connects to the backend API and socket server.
- Backend services manage authentication/session logic, interview orchestration, and persistence.
- MongoDB stores user/session/interview records; Atlas provides managed cloud hosting.
- Socket events synchronize prompts, answers, and live status updates during practice.

Project Structure

Path	Contents
backend/	models/, .env, .gitignore, package-lock.json, package.json
frontend/	src/, .env, .gitignore, index.html
logs/	server_err.log, server_out.log
QUICKSTART.md	file
README.md	file

APIs / Socket Events

- REST APIs commonly cover session creation, fetching interview data, saving responses, and retrieving history.
- Socket events typically include connect/disconnect, start interview, new question, submit answer, feedback, and end session.

- Event names and route paths should match the project's server implementation and frontend listeners.

Deployment Setup

- Vercel: host the frontend build and point it to the backend API base URL.
- Render: deploy the Node/Express backend as a web service with socket support.
- MongoDB Atlas: create a cluster, whitelist deployment IPs, and store the connection string securely.
- Use production environment variables in both platforms and keep secrets out of source control.

Environment Variables

- FRONTEND_API_URL or VITE_* API base URL for the deployed backend.
- MONGODB_URI for the Atlas connection string.
- PORT for backend runtime configuration.
- JWT_SECRET or session secret, if authentication is enabled.
- Any AI provider keys or webhook secrets required by the app, stored securely in hosting dashboards.

Closing Summary

- This project delivers a practical AI-powered interview rehearsal platform with a clear separation between client, server, sockets, and data storage.
- Its deployment model is well suited to Vercel, Render, and MongoDB Atlas for an accessible cloud-based practice experience.